PaTH-VLA: Progress-and-Target Context for Long-Horizon Robot Policy Adaptation

Anonymous CVPR submission

Abstract

Pretrained generalist robot policies fail on multi-stage kitchen tasks in two distinguishable ways: they lose track of where they are *in the task, and they act on the* wrong place. *Stage text and visual target prompts are popular remedies, but they are usually evaluated as complete inference-time systems, so it is unclear what a policy actually learns from them. We run a controlled, pre-registered ablation on three pretrained RoboCasa365 policies (Xiaomi Robotics 1, Awomo-0.5I and Awomo-0.5V). Each is adapted with rank-8 low-rank adapters for the same 1,000 updates under one of three auxiliary context signals: progress, i.e. predicting the current stage (S); target, i.e. seeing the target box and predicting its location (BL); or both, the full PaTH-VLA recipe (F). Target boxes are pseudo-labels from an open-vocabulary detector, a segmenter and a small vision-language model that picks the described instance, so no human draws a box. Every adapted policy is then evaluated exactly like the official checkpoint, with the original instruction and clean images, on 200 paired episodes over ten tasks, five of which were never seen during adaptation. The pre-registered contrasts are null in all three models: given one signal, adding the other changes success by at most $+3.5$ pp, and for Awomo-0.5V adding the box channel costs 8.5 pp ($p = 0.03$). Against the official checkpoints, every adapted XR-1 arm gains 6.5–7.5 pp, driven by the seen tasks, while both Awomo models lose up to 10–11 pp in the arms that saw boxes during training. Stage supervision is the harmless signal; box supervision can backfire when the policy's world-model pathway consumes the painted frame. We release the labelling pipeline, the protocol and all 2,400 logged episodes.*

1. Introduction

Generalist manipulation policies now ship with simulation benchmarks that score them on hundreds of multi-stage kitchen tasks [2, 20, 27]. On such tasks a policy fails for two reasons that are easy to confuse: it may not know *where it is* in the task (it re-opens a drawer it has already opened, or heads for the stove before the pan is in hand), or it may not know *where to look* (it reaches for the wrong container, or releases the pan beside the burner). A growing line of work addresses both with extra context: stage or subtask text produced by a planner [1, 4, 23], and visual prompts such as marks, boxes and sketched paths drawn into the image [8, 13, 16, 26, 28]. These methods are typically evaluated as complete systems, which makes it hard to tell what the policy actually learned from the additional context and what is merely supplied to it at test time.

This paper asks a narrower and, we think, more useful question. Starting from three pretrained policies that already score between 20% and 60% on ten RoboCasa365 tasks, we adapt each one for a fixed, small budget with lightweight adapters and one of three kinds of auxiliary context: *progress* supervision, i.e. predicting the current stage (S); *target* supervision, i.e. seeing the target box and predicting its location (BL); or both (F). We call the full recipe PaTH-VLA, for Progress-and-Target context, and treat S and BL as its two single-channel ablations. Then we evaluate every adapted policy exactly like the official checkpoint, with the original instruction and clean images, on 200 paired episodes per policy. Because the arms share the checkpoint, the data, the windows, the optimizer and the update count, the two pre-registered contrasts F$-$BL and F$-$S measure what each kind of supervision contributes to ordinary-input behaviour, not how well an inference-time prompting loop works.

Two design choices make the study practical. First, the supervision is cheap: stages come with the benchmark demonstrations, and target boxes are pseudo-labels produced by an open-vocabulary detector, a segmenter and a small vision-language model that picks the described instance among the detector's proposals (Figure 2); no human drew a box. Second, the protocol is paired and pre-registered: ten tasks, twenty seeds, four policies per model, the same initial scenes for all arms, and a decision rule fixed before the first update. We report every episode.

Our contributions are: (i) a controlled, cross-model ablation that separates stage supervision from box-and-location supervision for pretrained generalist policies on composite RoboCasa365 tasks, including five tasks never seen in adaptation; (ii) an automatic target-labelling pipeline

for subtask-annotated demonstrations that needs no human boxes and whose error profile we audit; and (iii) a pre-registered, paired evaluation protocol with 2,400 scored episodes whose outcome we state in Section 5. The outcome is a negative result with a clear shape: in none of the three models does either signal add to the other (all primary contrasts within ± 4 pp except one *negative* one), the stage-only arm is never significantly worse than the official checkpoint, and the arms that draw boxes during training carry every significant loss, up to -10 to -11 pp on the two world-action models.

2. Related Work

Generalist manipulation policies. Vision-language-action models map camera images, proprioception and a language instruction to action chunks after large-scale pretraining [5, 6, 12, 21–23, 25]. The three policies we adapt follow this recipe: XR-1 is a 5B vision-language-action model pretrained on more than 100K hours of real trajectories [27], and the two Awomo variants are world-action models that predict future image or video latents jointly with actions [2]. All three ship RoboCasa365 checkpoints [20], which lets us ask the same question across two model families with the official evaluation protocol.

Visual prompts and intermediate representations. Marks drawn into an image are an effective interface for vision-language models (Set-of-Mark [28], PIVOT [19]) and for robot policies: MOKA grounds keypoints from marks [16], ReKep optimizes keypoint constraints [11], RT-Trajectory and HAMSTER condition on sketched 2D paths [8, 13], and RT-Affordance predicts affordance overlays as an intermediate representation [18]. VP-VLA is closest to our box channel: it overlays a crosshair and a box produced by an external planner and segmenter on an additional prompted image and trains the controller with a visual-grounding loss [26]. We keep the prompted-image idea but ask a narrower causal question: given an already pretrained policy, does exposure to boxes during adaptation, together with a location head, change ordinary-input performance?

Stage and language hierarchies. Decomposing a long-horizon instruction into stages is a recurring idea, whether the stages are produced by a language model (SayCan [1], Code as Policies [14], VoxPoser [10]) or predicted by the policy itself as language motions or semantic subtasks (RT-H [4], $\pi_{0.5}$ [23]). RoboCasa365 demonstrations carry per-frame subtask annotations [20], which we use as a stage-classification target rather than as an input. Auxiliary prediction targets are known to speed up policy learning in navigation [29]; whether they help an already pretrained manipulation policy on composite kitchen tasks, and which target matters, is what we measure.

Parameter-efficient adaptation. We adapt with low-rank adapters [9] on a handful of late projections of the action transformer, so that all arms share the same trainable set and the same optimizer budget; the frozen perception and language backbones are never touched.

3. Method

3.1. Setting

A pretrained policy π_θ receives, at control step t, three RGB cameras I_t (left and right agent views and a wrist view, 256×256), a proprioceptive state $s_t \in \mathbb{R}^{16}$ and the task instruction ℓ, and outputs a chunk of 16 actions $a_{t:t+16} \in \mathbb{R}^{16\times12}$ for a mobile manipulator. We never change this interface at test time. During adaptation we have N demonstrations of seen tasks, each frame annotated with the official subtask index $z_t \in \{0, \dots, K-1\}$ and, after our labelling step, a normalized box $b_t^{(c)} \in [0,1]^4$ of the current subtask's target in each camera c (or "not visible").

3.2. Adapters and heads

We insert rank-8 low-rank adapters [9] into the last four attention projections of the policy's action transformer (query/key/value and output projections for XR-1, the analogous projections of the Awomo diffusion transformers), and keep every pretrained weight frozen. A forward hook captures the transformer's output feature $h_t \in \mathbb{R}^d$ (the first action token for XR-1, the mean over reference tokens for Awomo-0.5I, the mean over the last block's tokens for Awomo-0.5V). Two linear heads read h_t: a stage head $g_z(h_t) \in \mathbb{R}^{16}$ and a location head $g_b(h_t) \in [0,1]^{3\times2}$ that predicts the normalized centre of the target box in each camera. Both heads and the adapters are the only trainable parameters (1.2×10^5 for XR-1, 9.9×10^5 for Awomo-0.5I, 2.6×10^5 for Awomo-0.5V). Because the heads read the shared feature, their gradients flow into the adapters, i.e. auxiliary supervision can change the action pathway.

3.3. Losses and arms: progress, target, and PaTH-VLA

All arms keep the native action objective of their policy: a masked flow-matching regression [15] of the action chunk for XR-1, and the same regression plus the future image or video latent regression for the Awomo models. On top of it we define

$$\mathcal{L}_{\text{stage}} = \mathrm{CE}\big(g_z(h_t), z_t\big), \tag{1}$$

$$\mathcal{L}_{\text{loc}} = \sum_{c\,:\,\text{visible}} \mathrm{SmoothL1}\big(g_b(h_t)^{(c)}, \mathrm{centre}(b_t^{(c)})\big), \tag{2}$$

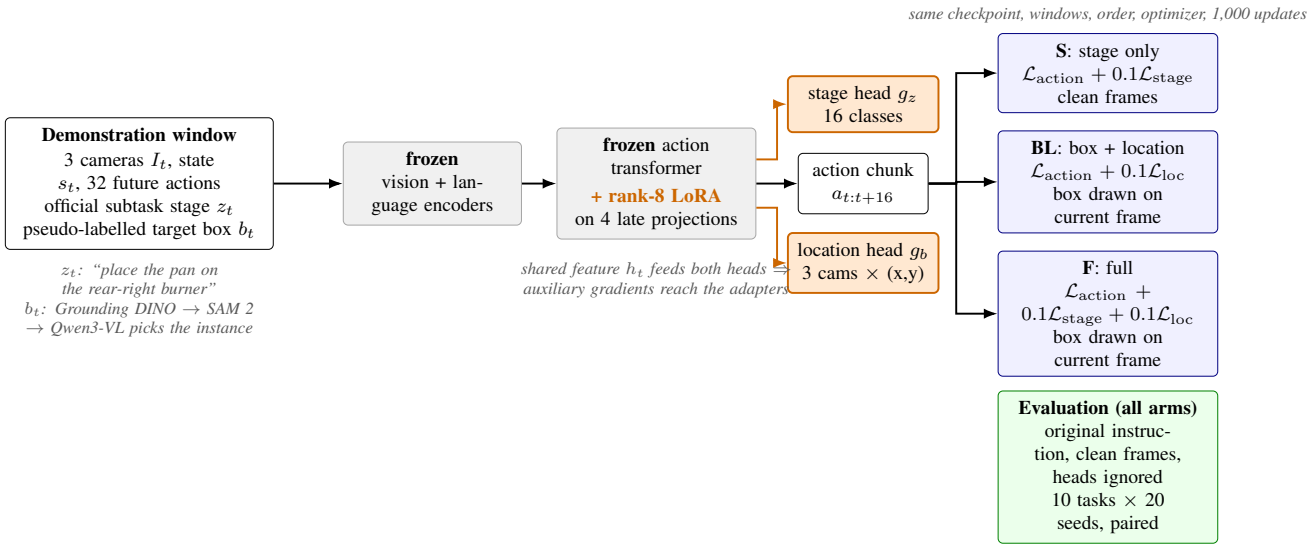


Figure 1. **Three ways of telling a pretrained policy where it is and where to look.** Left: a RoboCasa365 demonstration window (three cameras, state, actions) with its subtask stage and the pseudo-labelled target box. Middle: the frozen policy with rank-8 adapters on four late projections of its action transformer; a stage head and a location head read the same shared feature. Right: the three training arms. **S** adds only the stage loss; **BL** draws the target box on the current frame and adds the location loss; **F** does all three. Every arm is evaluated with ordinary inputs (no box, no stage text), so the comparison isolates what the extra supervision taught the policy.

and the total loss $\mathcal{L} = \mathcal{L}_{\text{action}} + 0.1\,\delta_S\,\mathcal{L}_{\text{stage}} + 0.1\,\delta_L\,\mathcal{L}_{\text{loc}}$, where $\delta_S, \delta_L \in \{0, 1\}$ switch the two auxiliary terms. The box channel draws a 2-pixel red rectangle around $b_t^{(c)}$ on the *current* frame of every camera in which the target is visible; history frames and future-prediction targets stay clean. The three arms are

- **S** (stage only): $\delta_S{=}1$, no box, $\delta_L{=}0$;
- **BL** (box + location): box drawn, $\delta_L{=}1$, $\delta_S{=}0$;
- **F** (full, PaTH-VLA): box drawn, $\delta_S{=}\delta_L{=}1$.

Within a model, S, BL and F start from the same checkpoint, see the same demonstration windows in the same order, and run the same number of optimizer updates with the same optimizer; only the switches above differ. No stage text is ever appended to the instruction, so the language input is identical across arms and identical to the official evaluation.

3.4. Target pseudo-labels

RoboCasa365 demonstrations contain subtask text and stage boundaries but no target boxes. We derive one target per stage description with a small rule table (the object to pick, the destination to place on or in, the part to actuate, the object to navigate to; 198 distinct descriptions in our five training tasks, none left undefined) and label every training window automatically: Grounding DINO [17] proposes boxes for the target's class name, SAM 2 [24] refines each proposal into a tight mask box, and Qwen3-VL-4B [3] selects, per camera, the proposal that is the described *instance* ("the tupperware with only the fish in it", "the knob for the front-left burner") or declares the target not visible, using set-of-mark prompting [28] on an upscaled copy of the frame. Windows whose target is visible in no camera, and "task complete" stages, are dropped for all arms, so the three arms train on exactly the same windows. No human corrected any label; Section 4 reports an audit of their quality.

3.5. Ordinary-input evaluation

At test time the adapted policy is called exactly like the official checkpoint: the original instruction, clean frames, native action chunking and the official success function. The heads are computed but ignored. This is deliberate: we want to know what the supervision taught the policy, not how well an inference-time prompting loop works. A feedback mode that redraws the policy's own box prediction is a separate, secondary experiment (Section 5).

4. Experimental Protocol

The protocol below was written down and hashed before the first optimizer update and before the first scored episode (pre-registration text in the supplement).

Models. XR-1 [27], Awomo-0.5I and Awomo-0.5V [2], each with its official RoboCasa365 checkpoint and native inference code wrapped in LeRobot [7]. The two Awomo models share a family, so the study covers three models from two families.



Figure 2. **Automatic target pseudo-labels** on six training windows (left, right and wrist cameras). The red box is the Grounding DINO → SAM 2 proposal that Qwen3-VL selected as the described instance; its centre is the location target. The pipeline resolves instance references (the container *with the carrot*, the knob of the *rear-right* burner, the *rear-right* burner itself) without any human input, and also makes typical mistakes: in the plate example the wrist view boxes a chair rather than the plate. No label was corrected by hand.

Tasks. Ten RoboCasa365 composite tasks, five from the composite-seen split (DeliverStraw, GetToastedBread, PackIdenticalLunches, SearingMeat, StirVegetables) and five from the composite-unseen split (ArrangeTea, CuttingToolSelection, RecycleBottlesByType, WaffleReheat, WeighIngredients). The ten were chosen before any run from the published per-task success rates, requiring every model to be between 20% and 60% on every task, so that both gains and losses are measurable; this is a performance-screened, not random, subset. The five unseen tasks are never used for training, labelling or tuning.

Training data and budget. Per seen task we fix 20 training and 5 development demonstrations by SHA-256 ranking of the official human demonstrations (100 + 25 episodes). Every anchor frame that is a multiple of 16 with at least 32 future frames forms a window (8,931 training windows); after labelling, 8,174 windows with a visible target remain and are shared by all arms. Each run performs 1,000 optimizer updates of AdamW (learning rate 10^{-4}, weight decay 0.01, one window per micro-batch, eight micro-batches per update, seed 20261106), i.e. 8,000 windows, about one pass over the panel. The update count was fixed from a timing calibration before launch and is identical for S, BL and F of every model.

Evaluation. Every policy (official checkpoint, S, BL, F; twelve in total) is evaluated on the same 200 episodes: ten tasks × twenty fixed seeds (0–19). A seed fully determines the kitchen layout, style, object instances and initial state (verified by hashing the first observation across processes), so the four policies of a model see identical scenes and the comparison is paired. We use the official per-task horizon, the official success function, native 16-step action chunks and a fixed policy RNG seed. All 2,400 episodes are reported; no task, seed or checkpoint was replaced after seeing results and only the final checkpoint of each run is evaluated.

Statistics. The two primary contrasts per model are F−BL (the contribution of stage supervision given box+location) and F−S (the contribution of box+location given stage). For each we report the paired success difference in percentage points pooled over the 200 pairs, a 95% interval from a task-stratified paired bootstrap (10,000 draws), an exact two-sided McNemar test on the discordant pairs, and Holm correction over the two primary contrasts of a model. We pre-specified that a contrast counts as supported only if its corrected interval excludes zero and the point estimate is at least +5 pp. The differences to the official checkpoint are secondary: they mix the effect of the

auxiliary supervision with that of additional action training, because, by design, no matched action-only arm was trained.

Label audit. A random sample of 48 labelled windows (144 camera views) was inspected after labelling. Of the 103 camera views with a visible-target label, 93 (90%) bound the described target (eight of them the whole appliance rather than the named part); the ten errors are six robot links, three wrong instances and one unrelated object (details in the supplement).

Compute. All runs use NVIDIA H100 PCIe GPUs; a training run takes 41 minutes for XR-1 and 107–173 minutes for the Awomo models, and an evaluation episode between one and ten minutes depending on the model and on whether it reaches the horizon.

5. Results

Primary contrasts: neither signal adds to the other. Table 2 and Figure 3b show the pre-registered contrasts. Adding stage supervision on top of box+location (F−BL) changes success by $+1.0$ pp for XR-1, $+3.5$ pp for Awomo-0.5I and -3.5 pp for Awomo-0.5V; every interval covers zero and the discordant pairs split almost evenly (28/26, 32/25, 29/36). Adding box+location on top of stage supervision (F−S) is $+1.0$ pp for XR-1 and -1.5 pp for Awomo-0.5I, again indistinguishable from zero, and -8.5 pp for Awomo-0.5V (95% CI $[-15.5, -1.5]$, 19 wins against 36 losses, exact $p = 0.030$; Holm-corrected $p = 0.060$, corrected interval $[-16.5, -1.0]$). Under the pre-registered rule no contrast is supported: within the budget we study, the stage head and the box-plus-location channel are not complementary, and for the video world-action model the box channel is harmful.

Against the official checkpoints: one model gains, two lose. For XR-1 all three arms are above the official checkpoint by a similar margin: $+6.5$ pp (S), $+6.5$ pp (BL) and $+7.5$ pp (F; CI $[+0.5, +14.5]$, 33 rescues against 18 regressions, $p = 0.049$). The gain comes from the seen tasks ($+13$ pp for F, CI $[+2, +24]$) and essentially vanishes on the five unseen tasks ($+2$ pp, CI $[-6, +11]$). Because the three arms differ only in their auxiliary signal yet gain alike, and because the gain tracks the tasks whose demonstrations were used, the most parsimonious reading is that XR-1 profits from the extra action regression on seen-task data rather than from either auxiliary signal; our design, which by choice has no action-only arm, cannot separate the two, and we do not claim it can. For the Awomo models the same adaptation is neutral at best. Awomo-0.5I drops by 5.0 (S), 10.0 (BL; CI $[-18.5, -1.5]$, $p = 0.033$) and 6.5 pp (F); the losses concentrate on the unseen tasks (BL: -18 pp, CI $[-29, -7]$, $p = 0.004$; F: -13 pp, $p = 0.029$) while the seen tasks are flat. Awomo-0.5V drops by 2.5 (S), 7.5 (BL) and 11.0 pp (F; CI $[-19.5, -3.0]$, $p = 0.015$), here on seen as well as unseen tasks.

Stage supervision is the safe signal; the box channel is not. Across the three models the stage-only arm is never significantly below the official checkpoint ($+6.5$, -5.0, -2.5 pp) and the two arms that draw boxes during training carry every significant loss (BL and F on Awomo-0.5I's unseen tasks, F on Awomo-0.5V). The models differ in how the painted frame enters the network: XR-1 encodes it with a frozen vision-language backbone whose tokens feed only the action transformer, whereas both Awomo models also regress a *clean* future image or video latent from a *painted* current frame, so the box becomes a train-time-only input to their world-model pathway and is absent at test time. The stage head, which reads an internal feature and adds nothing to the input, introduces no such mismatch. We state this as the explanation most consistent with the pattern, not as a tested mechanism.

How episodes fail. Every failure in all 2,400 episodes is a horizon timeout; no policy terminated early or triggered an infeasible state. Adapted XR-1 policies that succeed do so slightly faster (median 1,281 steps for F against 1,384 for the official checkpoint); for the Awomo models the successful-episode length does not change in a consistent direction. The per-task counts in Table 1 show that the losses of the Awomo arms are spread over many tasks rather than caused by one collapsed task, with the exception of Awomo-0.5V–F on PackIdenticalLunches (1/20 against 7/20), the task with the most instance-ambiguous labels ("the tupperware with only the fish in it"); Figure ef-fig:rollouts shows one such pair.

6. Limitations

Scope of the causal claim. The contrasts F−BL and F−S measure the value of one kind of supervision *given* the other, under one adaptation budget (1,000 updates, rank-8 adapters on four projections, one training seed). They do not establish that the effects are additive, and a different budget, adapter placement, loss weight or training seed could move them. We report one training seed per arm because the compute window did not allow three; the twenty evaluation seeds per task bound the evaluation noise but not the training noise.

No matched action-only arm. We compare adapted policies with the official checkpoint rather than with a policy adapted on the same windows without auxiliary losses.

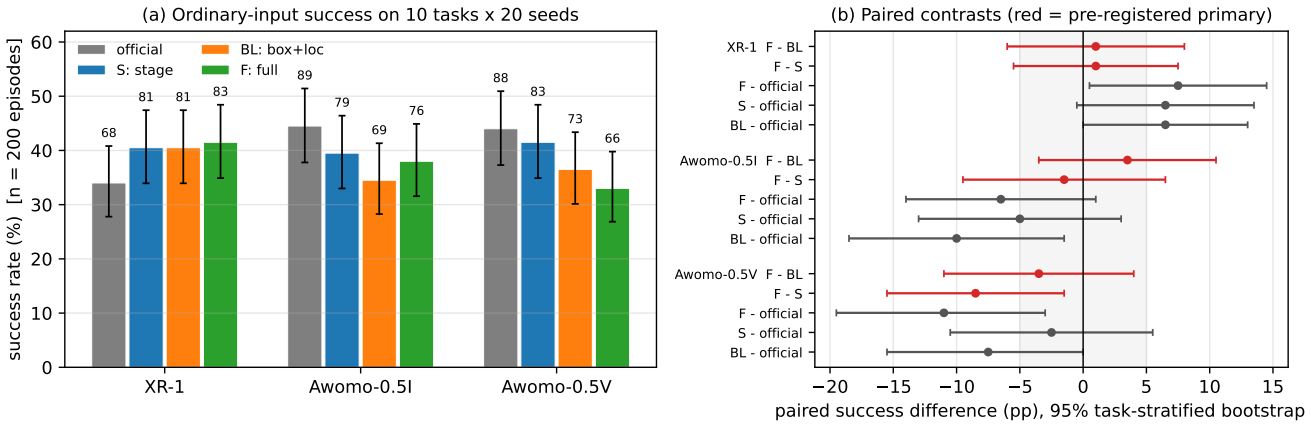


Figure 3. **Main result.** (a) Ordinary-input success of the official checkpoint and the three adapted arms of each model on the same 200 seeded episodes (10 tasks × 20 seeds); bars show the success count, whiskers the Wilson 95% interval. (b) Paired success differences with task-stratified paired-bootstrap 95% intervals; red: the two pre-registered primary contrasts per model; grey band: ±5 pp. No primary contrast is positive and significant in any model; the only corrected interval that excludes zero (F−S for Awomo-0.5V) is negative.

Model	Policy	Straw	Toast	Lunch	Sear	Stir	Tea	Cut	Recycle	Waffle	Weigh	Seen	Unseen	All
		composite-seen							composite-unseen					
XR-1	official	10	2	5	5	6	10	9	1	12	8	28	40	**34.0**
	S	7	5	8	9	8	14	12	0	12	6	37	44	**40.5**
	BL	7	6	8	7	9	9	13	3	10	9	37	44	**40.5**
	F	10	6	8	9	8	9	11	2	12	8	41	42	**41.5**
Awomo-0.5I	official	12	6	5	4	10	10	12	6	12	12	37	52	**44.5**
	S	13	6	5	8	6	11	9	2	10	9	38	41	**39.5**
	BL	14	4	9	4	4	8	11	3	6	6	35	34	**34.5**
	F	10	4	9	6	8	7	9	4	9	10	37	39	**38.0**
Awomo-0.5V	official	13	10	7	6	8	13	7	9	7	8	44	44	**44.0**
	S	9	7	10	4	12	11	6	8	9	7	42	41	**41.5**
	BL	8	5	8	4	9	12	3	8	8	8	34	39	**36.5**
	F	10	6	1	4	9	8	5	4	9	10	30	36	**33.0**

Table 1. **Successes out of 20 seeded episodes per task**, with seen, unseen and overall success rates (%) for the official checkpoint and the three adapted arms of each model. All arms are evaluated with ordinary inputs on the same 200 initial scenes.

Differences to the official checkpoint therefore include the effect of additional action-regression training on seen-task demonstrations and must not be read as the benefit of the auxiliary supervision alone; only the within-model contrasts isolate it.

Pseudo-labels. Target boxes come from an automatic pipeline whose audited error rate is reported in Section 4: it sometimes boxes a robot link or the wrong instance among duplicates, and the location target is a 2D box centre, not a contact point. Our conclusions about the box channel are conclusions about *this* label quality.

Benchmark and task selection. The ten tasks were screened by published success rate; results on them do not transfer to the whole benchmark, and the composite-unseen tasks test transfer of what was learned on five seen tasks, not open-world generalization. Simulation only; no real-robot evaluation.

7. Conclusion

With PaTH-VLA we asked what pretrained generalist policies learn from two cheap kinds of context, where they are in a task and where they should look, by adapting three RoboCasa365 policies with matched budgets and evaluating them with ordinary inputs on the same 200 scenes. We found no evidence that the two signals are comple-

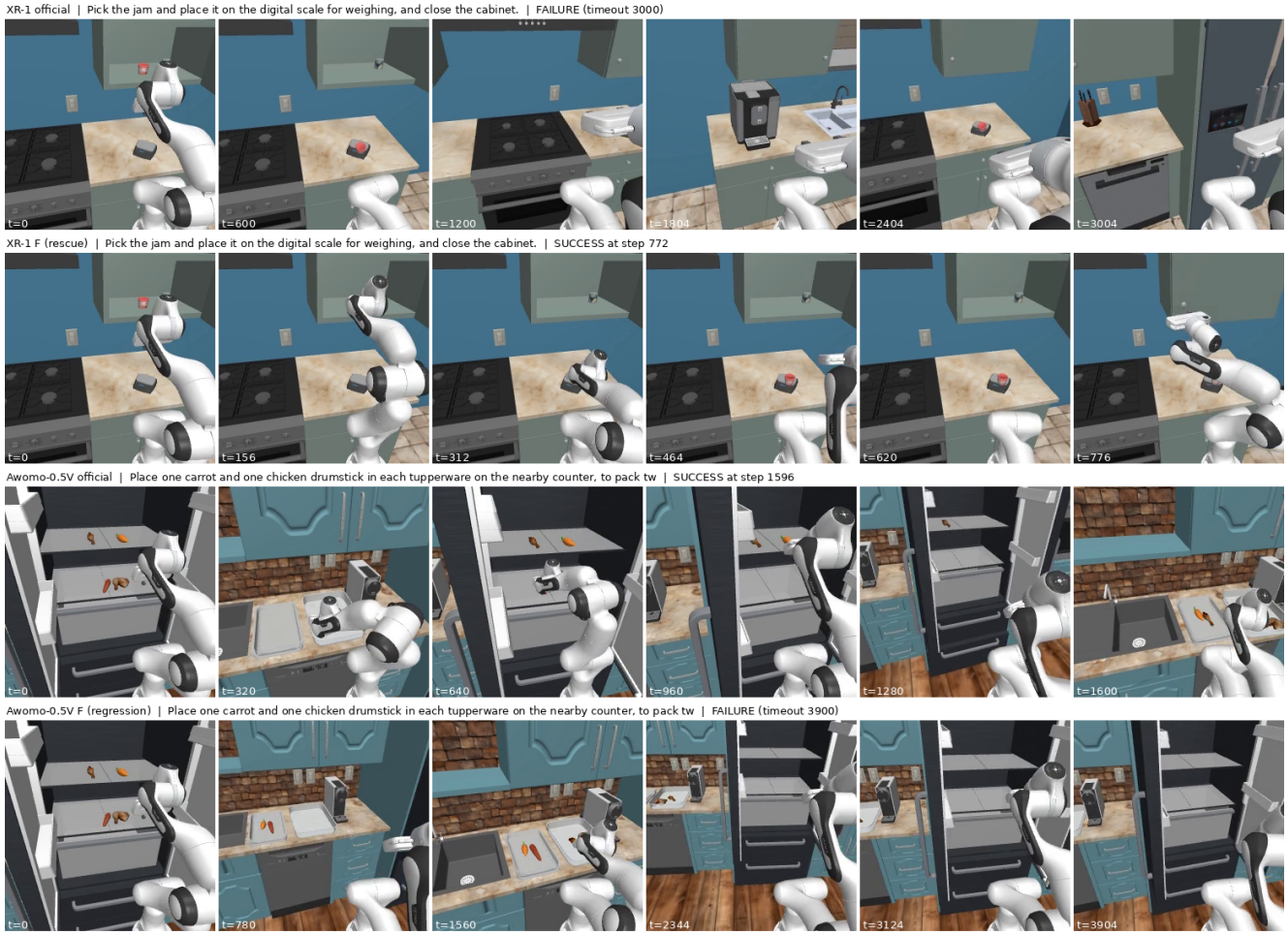


Figure 4. **Qualitative rollouts** (agent view, six evenly spaced frames). Top: WeighIngredients, seed 12. The official XR-1 wanders between the stove, the sink and the fridge until the horizon; the F-adapted policy picks the jam and places it on the scale at step 772. Bottom: PackIdenticalLunches, seed 17. The official Awomo-0.5V completes the task at step 1,596; its F-adapted version carries food to the counter but then drifts to the wrong drawers and times out. Both pairs start from identical scenes.

mentary, a consistent small gain for XR-1 that we cannot attribute to either signal, and a consistent loss for the world-action models whenever boxes were drawn into their training frames. The practical lesson is asymmetric: a stage head is a cheap and harmless addition, whereas visual prompts that are present at training time and absent at test time are a liability for policies that predict future frames. The labelling pipeline, the pre-registered protocol and every logged episode are released so that the same question can be asked of the next generation of policies with a larger adaptation budget and more training seeds.

Use of large language models

Large language models (OpenAI and Anthropic assistants) were used as coding and writing agents under the authors' direction: they implemented the labelling and evaluation scripts, launched and monitored the runs, drafted text and figures, and compiled references. Qwen3-VL-4B is part of the labelling pipeline described in Section 3. All experimental decisions, the pre-registered protocol, and the final claims are the authors' responsibility; every number in the paper is computed from the logged episodes by the released analysis script.

References

[1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. Do as I can, not as I say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*, pages 287–318, 2022. 1, 2

[2] Awomo-WAM Team. Awomo-0.5: World action model pre-trained at scale. https://huggingface.co/Auwomo, 2026. Image (Awomo-0.5I) and video (Awomo-0.5V) variants; RoboCasa365 check-

Model	Contrast	Δ (pp)	95% CI	W/L	p
XR-1	**F−BL**	+1.0	[-6.0, +8.0]	28/26	0.892
	F−S	+1.0	[-5.5, +7.5]	25/23	0.885
	F−official	+7.5	[+0.5, +14.5]	33/18	0.049
	S−official	+6.5	[-0.5, +13.5]	33/20	0.098
	BL−official	+6.5	[+0.0, +13.0]	31/18	0.085
Awomo-0.5I	**F−BL**	+3.5	[-3.5, +10.5]	32/25	0.427
	F−S	-1.5	[-9.5, +6.5]	33/36	0.810
	F−official	-6.5	[-14.0, +1.0]	24/37	0.124
	S−official	-5.0	[-13.0, +3.0]	33/43	0.302
	BL−official	-10.0	[-18.5, -1.5]	30/50	0.033
Awomo-0.5V	**F−BL**	-3.5	[-11.0, +4.0]	29/36	0.457
	F−S	-8.5	[-15.5, -1.5]	19/36	0.030
	F−official	-11.0	[-19.5, -3.0]	27/49	0.015
	S−official	-2.5	[-10.5, +5.5]	31/36	0.625
	BL−official	-7.5	[-15.5, +0.0]	26/41	0.086

Table 2. **Paired contrasts** pooled over 200 (task, seed) pairs per model: success-rate difference, task-stratified paired-bootstrap 95% interval, wins/losses among discordant pairs, exact McNemar p. Bold rows are the two pre-registered primary contrasts; the Holm-corrected verdicts are in the text.

points `Auwomo/Awomo-0.5I-RoboCasa365` and `Auwomo/Awomo-0.5V-RoboCasa365`; technical report not yet released at the time of writing. 1, 2, 3

[3] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, et al. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025. Qwen Team, Alibaba Group. 3

[4] Suneel Belkhale, Tianli Ding, Ted Xiao, Pierre Sermanet, Quon Vuong, Jonathan Tompson, Yevgen Chebotar, Debidatta Dwibedi, and Dorsa Sadigh. RT-H: Action hierarchies using language. In *Robotics: Science and Systems (RSS)*, 2024. 1, 2

[5] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: A vision-language-action flow model for general robot control. In *Robotics: Science and Systems (RSS)*, 2025. 2

[6] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, pages 2165–2183, 2023. 2

[7] Remi Cadene, Simon Alibert, Alexander Soare, Quentin Gallouedec, Adil Zouitine, Steven Palma, Pepijn Kooijmans, Michel Aractingi, Mustafa Shukor, Dana Aubakirova, Martino Russi, Francesco Capuano, Caroline Pascal, Jade Choghari, Khalil Meftah, Maxime Ellerbach, Jess Moss, and Thomas Wolf. LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch. https://github.com/huggingface/lerobot, 2024. 3

[8] Jiayuan Gu, Sean Kirmani, Paul Wohlhart, Yao Lu, Montserrat Gonzalez Arenas, Kanishka Rao, Wenhao Yu, Chuyuan Fu, Keerthana Gopalakrishnan, Zhuo Xu, Priya Sundaresan, Peng Xu, Hao Su, Karol Hausman, Chelsea Finn, Quan Vuong, and Ted Xiao. RT-Trajectory: Robotic task generalization via hindsight trajectory sketches. In *International Conference on Learning Representations (ICLR)*, 2024. 1, 2

[9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. 2

[10] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. VoxPoser: Composable 3D value maps for robotic manipulation with language models. In *Conference on Robot Learning (CoRL)*, pages 540–562, 2023. 2

[11] Wenlong Huang, Chen Wang, Yunzhu Li, Ruohan Zhang, and Li Fei-Fei. ReKep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation. In *Conference on Robot Learning (CoRL)*, pages 4573–4602, 2024. 2

[12] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, pages 2679–2713, 2024. 2

[13] Yi Li, Yuquan Deng, Jesse Zhang, Joel Jang, Marius Memmel, Raymond Yu, Caelan Reed Garrett, Fabio Ramos, Dieter Fox, Anqi Li, Abhishek Gupta, and Ankit Goyal. HAMSTER: Hierarchical action models for open-world robot manipulation. In *International Conference on Learning Representations (ICLR)*, 2025. 1, 2

[14] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023. 2

[15] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023. 2

[16] Fangchen Liu, Kuan Fang, Pieter Abbeel, and Sergey Levine. MOKA: Open-world robotic manipulation through mark-based visual prompting. In *Robotics: Science and Systems (RSS)*, 2024. 1, 2

[17] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *European Conference on Computer Vision (ECCV)*, 2024. 3

[18] Soroush Nasiriany, Sean Kirmani, Tianli Ding, Laura Smith, Yuke Zhu, Danny Driess, Dorsa Sadigh, and Ted

Xiao. RT-Affordance: Affordances are versatile intermediate representations for robot manipulation. *arXiv preprint arXiv:2411.02704*, 2024. 2

[19] Soroush Nasiriany, Fei Xia, Wenhao Yu, Ted Xiao, Jacky Liang, Ishita Dasgupta, Annie Xie, Danny Driess, Ayzaan Wahid, Zhuo Xu, Quan Vuong, Tingnan Zhang, Tsang-Wei Edward Lee, Kuang-Huei Lee, Peng Xu, Sean Kirmani, Yuke Zhu, Andy Zeng, Karol Hausman, Nicolas Heess, Chelsea Finn, Sergey Levine, and Brian Ichter. PIVOT: Iterative visual prompting elicits actionable knowledge for VLMs. In *International Conference on Machine Learning (ICML)*, pages 37321–37341, 2024. 2

[20] Soroush Nasiriany, Sepehr Nasiriany, Abhiram Maddukuri, and Yuke Zhu. RoboCasa365: A large-scale simulation framework for training and benchmarking generalist robots. In *International Conference on Learning Representations (ICLR)*, 2026. 1, 2

[21] NVIDIA, Johan Bjorck, et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. 2

[22] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, You Liang Tan, Lawrence Yunliang Chen, Pannag Sanketi, Quan Vuong, Ted Xiao, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems (RSS)*, 2024.

[23] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 1, 2

[24] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations (ICLR)*, 2025. 3

[25] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. 2

[26] Zixuan Wang, Yuxin Chen, Yuqi Liu, Jinhui Ye, Pengguang Chen, Changsheng Lu, Shu Liu, Bei Yu, and Jiaya Jia. VP-VLA: Visual prompting as an interface for vision-language-action models. *arXiv preprint arXiv:2603.22003*, 2026. 1, 2

[27] Xiaomi Robotics Team. Xiaomi-Robotics-1: Scaling vision-language-action models with over 100k hours of real-world trajectories. *arXiv preprint arXiv:2607.15330*, 2026. Model checkpoints: `https://huggingface.co/XiaomiRobotics/Xiaomi-Robotics-1-5B`; referred to as XR-1 on the model card. 1, 2, 3

[28] Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. Set-of-mark prompting unleashes extraordinary visual grounding in GPT-4V. *arXiv preprint arXiv:2310.11441*, 2023. 1, 2, 3

[29] Joel Ye, Dhruv Batra, Erik Wijmans, and Abhishek Das. Auxiliary tasks speed up learning PointGoal navigation. In *Conference on Robot Learning (CoRL)*, pages 498–516, 2020. 2